Lecture 3: Memory Management and File Systems in Linux

Duration: 1 Hour
Objective: Understand memory management and file systems in Linux for HPC applications.
Topics Covered:

- Memory Management: Virtual memory, paging, swapping, memory allocation in Linux.
- File Systems: Types (ext4, NFS, Lustre), their role in HPC, and file system hierarchy.

Introduction

Welcome to Lecture 3 on Memory Management and File Systems in Linux. Today, we'll dive into how Linux manages memory and organizes file systems, with a focus on their importance in High-Performance Computing (HPC). In HPC, where massive computations and parallel processes are common, efficient memory and file system management are key to achieving top performance. We'll cover each topic with simple examples to make these concepts clear and relevant.

Memory Management in Linux

Memory management ensures that processes share the system's memory efficiently while staying isolated. This is especially critical in HPC, where many processes run across multiple nodes.

What is Virtual Memory?

Virtual memory is a memory management technique used by operating systems to give each process its own **private address space**. This creates the illusion that the process has exclusive access to the entire memory, even though it is sharing the physical memory (RAM) with other processes. The operating system (OS) achieves this by assigning each process a set of **virtual addresses**, which are then mapped to **physical addresses** in RAM or, if necessary, to disk storage (known as swap space).

How Does It Work?

Virtual memory relies on a mechanism called **paging**:

- The virtual address space of a process is divided into fixed-size units called **pages** (e.g., 4 KB).

- The physical memory is similarly divided into **frames** of the same size.
- A **page table**, maintained by the OS, keeps track of the mapping between virtual pages and physical frames.
- When a process accesses a virtual address, the **Memory Management Unit (MMU)** translates it to a physical address using the page table.
- If the requested page is not in RAM (i.e., it has been swapped out to disk), a **page fault** occurs. The OS then retrieves the page from disk, loads it into RAM, and updates the page table.

This abstraction allows the system to manage memory efficiently and support multiple processes simultaneously.

Benefits of Virtual Memory

Virtual memory provides two primary advantages: **isolation** and **flexibility**.

1. Isolation

- Each process operates within its own virtual address space, meaning it cannot directly access the memory of other processes.
- This isolation enhances **security** by preventing unauthorized access to sensitive data and improves **stability** by ensuring that a malfunctioning process cannot corrupt the memory of others.
- In **HPC environments**, where multiple parallel tasks (e.g., simulations or data analyses) run concurrently, isolation is critical to prevent interference and ensure consistent, reliable results.

2. Flexibility

- Virtual memory allows the system to run more processes than the physical memory (RAM) can accommodate by using **disk space** as an extension of RAM.
- When RAM is full, less-used pages are swapped out to disk, freeing up space for other processes. This enables the system to handle larger workloads or more concurrent processes than would otherwise be possible.
- In HPC, this flexibility supports running complex, memory-intensive applications that may collectively exceed the available physical memory.

Virtual Memory in HPC

In **High-Performance Computing (HPC)**, virtual memory plays a vital role in managing the memory demands of multiple parallel tasks. HPC systems often involve:

- Executing numerous processes or threads simultaneously across compute nodes.
- Processing large datasets that may exceed the physical memory of individual nodes.
- Ensuring that parallel tasks operate independently without memory conflicts.

Virtual memory provides the necessary isolation to prevent tasks from interfering with each other's memory spaces and the flexibility to run more tasks than the physical RAM can support. However, excessive swapping between RAM and disk (known as **thrashing**) can degrade performance, which is a critical concern in HPC where speed is paramount. To mitigate this, HPC systems are typically equipped with large amounts of RAM to minimize reliance on swap space.

Example: Running Two HPC Jobs with Limited RAM

Let's explore how virtual memory works with a practical example in an HPC context.

Scenario

- A system has **3 GB of physical RAM** available.
- There are **two HPC jobs**, each requiring **2 GB of memory**.
- Without virtual memory, running both jobs simultaneously would be impossible because the total memory required (2 GB + 2 GB = 4 GB) exceeds the available RAM (3 GB).

How Virtual Memory Solves This

1. **Virtual Address Spaces**:
   - Each job is assigned its own virtual address space of 2 GB. From the perspective of each job, it has exclusive access to a full 2 GB of memory, unaware of the other job or the physical memory constraints.
2. **Memory Mapping**:
   - The OS maps the virtual addresses of each job to the available physical RAM. Since the total virtual memory (4 GB) exceeds the physical RAM (3 GB), not all pages can reside in RAM at once.
3. **Paging and Swapping**:
   - The OS loads the actively used pages of each job into RAM. For instance:

        - It might allocate 1.5 GB of RAM to each job, totaling 3 GB (the full capacity of the RAM).
        - The remaining 0.5 GB of each job's memory (1 GB total) is stored in the swap space on disk.
    - When a job needs to access a page that is not in RAM, a **page fault** occurs. The OS swaps out an inactive page from RAM to disk and swaps in the required page from disk, updating the page table accordingly.
4. **Isolation in Action**:
    - Even if both jobs access the same virtual address (e.g., 0x1000), these addresses map to different physical locations in RAM or disk. This ensures that the jobs do not interfere with each other, maintaining data integrity and security.
5. **Outcome**:
    - With virtual memory, both jobs can run concurrently without crashing, despite the system having only 3 GB of RAM. The OS dynamically manages which pages are kept in RAM and which are swapped to disk based on the jobs' needs.
    - However, if both jobs frequently access their full 2 GB of memory, excessive swapping could occur, slowing down execution. In HPC, this is often avoided by providing sufficient RAM or optimizing the jobs to use memory more efficiently.

Visual Representation

Here's a simplified view of the memory allocation:

Component	Job 1	Job 2
Virtual Memory	2 GB	2 GB
In RAM	1.5 GB	1.5 GB
In Swap Space	0.5 GB	0.5 GB

- **Total RAM Used**: 3 GB (1.5 GB + 1.5 GB).
- **Total Swap Used**: 1 GB (0.5 GB + 0.5 GB).

This demonstrates how virtual memory leverages both RAM and disk to accommodate the 4 GB total requirement with only 3 GB of physical memory.

Key Takeaways

- **Virtual Memory**: Gives each process its own private address space, making it seem like it has the entire memory to itself.

- **Isolation**: Prevents processes from accessing each other's memory, enhancing security and stability—crucial for HPC's parallel tasks.
- **Flexibility**: Allows the system to run more processes than the physical RAM can hold by using disk space, as shown in the example.
- **HPC Relevance**: Supports running multiple parallel tasks without interference, though performance depends on minimizing swapping.

In summary, virtual memory is a cornerstone of modern operating systems, enabling efficient memory management and supporting complex workloads like those in HPC. The example of two 2-GB jobs running on a 3-GB RAM system illustrates how it maps virtual addresses to available RAM or disk, ensuring both jobs can execute without crashing.

2. Paging

Paging splits virtual memory into fixed-size **pages** (e.g., 4 KB) and physical memory into **frames** of the same size. A **page table** maps virtual pages to physical frames, allowing non-contiguous memory use.

In HPC, paging optimizes memory for large datasets and multiple processes.

Example:
A process needs 12 KB of memory, split into three 4 KB pages. The page table might map:

- Page 0 → Frame 1
- Page 1 → Frame 4
- Page 2 → Frame 6
  This scatters the data across physical memory but looks contiguous to the process.

Explaining Paging: Optimizing Memory for Large Datasets and Multiple Processes

Paging is a memory management technique used by operating systems to make the most of available physical memory (RAM) when handling large datasets and running multiple processes at once. It works by dividing memory into small, fixed-size pieces, allowing processes to use memory efficiently even when it's scattered across different locations. Below, I'll explain how paging works in detail, using a simple example to show how it optimizes memory usage.

What Is Paging?

Paging splits both the **virtual memory** (the memory a process thinks it has) and the **physical memory** (the actual RAM) into fixed-size units:

- **Pages**: Chunks of virtual memory used by a process.
- **Frames**: Chunks of physical memory in RAM, the same size as pages.

The operating system uses a **page table** to keep track of where each page of a process's virtual memory is stored in physical memory frames. This allows the process's memory to be scattered across different locations in RAM while still appearing as one continuous block to the process itself.

Example: A Process Needing 12 KB of Memory

Let's dive into the example provided:

- A process needs **12 KB** of memory.
- The system uses a page size of **4 KB**.
- This means the process's 12 KB of memory is split into **three pages**:
  - **Page 0**: The first 4 KB (addresses 0 to 4095).
  - **Page 1**: The next 4 KB (addresses 4096 to 8191).
  - **Page 2**: The final 4 KB (addresses 8192 to 12287).

Physical memory (RAM) is also divided into **frames** of 4 KB each. The operating system assigns these pages to available frames using the page table. In this example, the page table maps:

- **Page 0 → Frame 1**
- **Page 1 → Frame 4**
- **Page 2 → Frame 6**

This means:

- The first 4 KB of the process (Page 0) is stored in Frame 1.
- The next 4 KB (Page 1) is stored in Frame 4.
- The last 4 KB (Page 2) is stored in Frame 6.

These frames—1, 4, and 6—don't need to be next to each other in physical memory. They could be scattered anywhere in RAM, depending on what's available.

Here's a simple picture of what this might look like in physical memory:

```
Frame 0: [Data from another process]
Frame 1: [Page 0 of our process]
Frame 2: [Data from another process]
Frame 3: [Empty or other data]
Frame 4: [Page 1 of our process]
Frame 5: [Data from another process]
Frame 6: [Page 2 of our process]
...
```

Even though the pages are scattered across Frames 1, 4, and 6, the process doesn't notice. To the process, its 12 KB of memory looks like one continuous block, thanks to the page table.

How Does the Process Access Its Memory?

When the process wants to use a specific memory address, the operating system steps in:

- Suppose the process accesses address **5000** (5 KB into its memory).
- Since each page is 4 KB:
  - Addresses 0–4095 are in Page 0 (mapped to Frame 1).
  - Addresses 4096–8191 are in Page 1 (mapped to Frame 4).
  - Addresses 8192–12287 are in Page 2 (mapped to Frame 6).
- Address 5000 falls in Page 1 (because it's between 4096 and 8191).
- The page table says Page 1 is in Frame 4, so the OS fetches the data from Frame 4, adjusting the offset (5000 - 4096 = 904 bytes into Frame 4).

This translation happens behind the scenes, so the process can work as if its memory is all in one place.

Why Does Paging Optimize Memory?

Now, let's explore why this setup is so great for handling **large datasets** and **multiple processes**.

1. Efficient Use of Scattered Memory

- In a busy system, free memory might be broken into small chunks rather than one big block. For example, after running several programs, the only free frames might be 1, 4, and 6, with other frames in use.
- Without paging, a process needing 12 KB would need one continuous 12 KB block, which might not exist. Paging lets it use three separate 4 KB frames instead, making better use of whatever memory is available.
- For **large datasets**—like a scientific simulation needing gigabytes of memory—paging ensures the system can piece together enough scattered frames, even if no single huge block is free.

2. Reduced Fragmentation

- Over time, as processes start and stop, memory can get fragmented—lots of small free spaces, but none big enough for a new process. This is called **external fragmentation**.
- Paging fixes this by working with fixed-size frames. Any free 4 KB frame can hold any 4 KB page, so small gaps don't go to waste. This is a big win for systems running large programs or many processes.

3. Running Multiple Processes

- Paging lets several processes share physical memory at the same time. While our process uses Frames 1, 4, and 6, another process might use Frames 0, 2, and 3.
- The operating system can quickly switch between processes by updating which page tables are active, making it easier to multitask. This is especially important in environments like servers or high-performance computing (HPC), where many tasks run together.

4. Support for Large Datasets with Virtual Memory

- Paging is the foundation of **virtual memory**, which lets processes use more memory than the system physically has by swapping pages to disk when RAM is full.
- For example, if our 12 KB process runs on a system with only 8 KB of free RAM, the OS might keep Pages 0 and 1 in Frames 1 and 4, while Page 2 waits on disk. When the process needs Page 2, the OS swaps it into a free frame. This trick is crucial for large datasets that exceed available RAM.

A Simple Analogy

Imagine physical memory as a **bookshelf** with separate shelves (frames), and the process's memory as a **book** split into chapters (pages). The chapters don't need to be on the same shelf—they might be on Shelf 1, Shelf 4, and Shelf 6. As long as you have a **directory** (the page table)

telling you where each chapter is, you can still read the book in order. Paging works the same way: the process reads its memory like a single book, even though the pages are scattered across the bookshelf of RAM.

3. Swapping

Swapping moves inactive pages from RAM to disk (swap space) when RAM is full. It lets more processes run but slows things down because disk access is slower than RAM.

In HPC, too much swapping hurts performance, so systems often have lots of RAM to avoid it.

Example:
A node with 4 GB RAM runs jobs needing 6 GB. The OS swaps 2 GB to disk, keeping the jobs running, but data access slows down when swapped pages are needed.

1.
   - for disk operations to complete.

Visualizing the Process

- **Before Swapping**:
  - RAM: 4 GB (full, but jobs need 6 GB → not enough).
  - Swap Space: 0 GB.
- **After Swapping**:
  - RAM: 4 GB (active data).
  - Swap Space: 2 GB (inactive data).
- **When Accessing Swapped Data**:
  - Swap out: Move an unneeded page from RAM to disk.
  - Swap in: Bring the needed page from disk to RAM.
  - Result: The job proceeds, but with a noticeable delay.

Impact on Performance

In this example, the node with 4 GB of RAM can run jobs needing 6 GB by swapping 2 GB to disk. However:

- **If swapping is rare**: The slowdown might be minimal, and the system can still function effectively.
- **If swapping is frequent**: Constantly moving pages between RAM and disk (thrashing) will degrade performance significantly, as the CPU waits for data instead of computing.

In an HPC context, where speed is essential, frequent swapping would undermine the system's ability to deliver fast results. This is why HPC nodes are often provisioned with enough RAM to handle their workloads without relying on swap space.

4. Memory Allocation in Linux

Linux uses the **buddy system** to allocate memory, splitting blocks into powers of two (e.g., 64 KB, 128 KB). It's fast and efficient for large allocations, which HPC apps often need.

Example:
A job requests 200 KB. The buddy system allocates a 256 KB block, splits it into two 128 KB blocks, and gives one to the job, saving the other for later.

Memory Allocation in Linux: Overview

In Linux, memory allocation is a critical task managed by the kernel to ensure that processes, including user applications and system operations, have the memory they need to function. One of the key mechanisms Linux uses for managing physical memory is the **buddy system**, especially for allocating large, contiguous blocks of memory.

What is the Buddy System?

The **buddy system** is a memory allocation algorithm that organizes memory into blocks whose sizes are **powers of two** (e.g., 64 KB, 128 KB, 256 KB, 512 KB, etc.). It's designed to efficiently allocate and deallocate memory while minimizing fragmentation, making it particularly suitable for scenarios where large memory blocks are required.

How the Buddy System Works:

- **Block Sizes**: Memory is divided into blocks of sizes 2^k, where k is an integer (e.g., 2^6 = 64 KB, 2^7 = 128 KB, etc.).
- **Allocation**: When a process requests memory of size S, the system finds the smallest block size 2^k such that $2^k \geq S$.

- **Splitting**: If no block of the exact size is available, the system takes a larger block, splits it into two equal "buddies," and repeats this process until a block of the appropriate size is obtained.
- **Merging**: When a block is freed, the system checks if its "buddy" (the other half of the original split) is also free. If so, it merges them into a larger block to reduce fragmentation.

This approach makes allocation and deallocation fast because it relies on a simple, hierarchical structure rather than searching through fragmented memory regions.

Efficiency for Large Allocations and Relevance to HPC

The buddy system shines in scenarios requiring **large, contiguous memory allocations**, which are common in **High-Performance Computing (HPC)** applications. HPC workloads—such as scientific simulations, machine learning training, or large-scale data analysis—often need vast amounts of memory for:

- Storing large datasets (e.g., climate models or genomic data).
- Performing computations on massive arrays or matrices.
- Supporting parallel processes that share memory.

Why is it Efficient for HPC?

- **Speed**: The buddy system quickly identifies and allocates large blocks by working with powers of two, avoiding the overhead of searching through many small, scattered free spaces.
- **Contiguity**: It provides contiguous memory blocks, which improves cache efficiency and reduces memory access latency—key factors in HPC performance.
- **Fragmentation Control**: By merging freed blocks with their buddies, it prevents external fragmentation, ensuring that large blocks remain available for future allocations.

In contrast, other allocation methods might struggle to provide large contiguous regions or could slow down due to fragmentation, making the buddy system a preferred choice for HPC environments.

Analyzing and Correcting the Provided Example

The query provides an example:
"A job requests 200 KB. The buddy system allocates a 256 KB block, splits it into two 128 KB blocks, and gives one to the job, saving the other for later."

This example contains a logical inconsistency:

- The job requests **200 KB**.
- A **128 KB block** is smaller than 200 KB, so allocating a 128 KB block would not satisfy the request.
- The buddy system typically allocates a single contiguous block large enough for the request, not a smaller block after splitting.

Let's correct this and provide a clear explanation.

Correcting the Example

For a **200 KB request**, here's how the buddy system actually works, assuming block sizes are powers of two (e.g., 128 KB, 256 KB, 512 KB):

1. **Determine the Required Block Size**:
   - The request is 200 KB.
   - Since 128 KB < 200 KB and 256 KB > 200 KB, the smallest suitable block is **256 KB**.
2. **Allocation**:
   - If a **256 KB block** is available, the system allocates it directly to the job.
   - The job gets 256 KB, which is more than the 200 KB requested, but this is the smallest power of two that meets the need.
3. **If No 256 KB Block is Available**:
   - Suppose only a **512 KB block** is available.
   - The system splits the 512 KB block into two **256 KB buddies**.
   - It allocates one 256 KB block to the job and keeps the other 256 KB block free.
4. **Outcome**:
   - The job receives a **256 KB block**, satisfying its 200 KB request.
   - No 128 KB blocks are involved because 128 KB is insufficient for 200 KB.

Why the Original Example is Misleading

The original example suggests splitting a 256 KB block into two 128 KB blocks for a 200 KB request. This wouldn't work because:

- 128 KB < 200 KB, so the allocated block wouldn't meet the job's needs.

- The buddy system doesn't split a block into pieces smaller than the request size and then allocate one piece; it allocates a single block that's large enough.

A plausible scenario where splitting into 128 KB blocks makes sense would be if the request were smaller, e.g., **100 KB**. Let's explore that as a revised example.

Revised Example: Step-by-Step

Let's assume the system manages memory in blocks of sizes 64 KB, 128 KB, 256 KB, 512 KB, etc., and a job requests **100 KB** (a size that aligns with the splitting behavior described).

Scenario

- Job requests: **100 KB**.
- Available block sizes: 64 KB, 128 KB, 256 KB, etc.

Allocation Process

1. **Determine the Required Block Size**:
   - The request is 100 KB.
   - Since 64 KB < 100 KB and 128 KB > 100 KB, the smallest suitable block is **128 KB**.
2. **Check Availability**:
   - If a **128 KB block** is available, the system allocates it directly to the job.
3. **Splitting (if Necessary)**:
   - If no 128 KB block is available, but a **256 KB block** is, the system:
     - Splits the 256 KB block into two **128 KB buddies**.
     - Allocates one 128 KB block to the job.
     - Keeps the other 128 KB block free for future use.
4. **Outcome**:
   - The job gets a **128 KB block**, which satisfies its 100 KB request.
   - The remaining 128 KB block is saved for later.

Visual Representation

- **Before Splitting**:

```
[256 KB free]
```

- **After Splitting and Allocation**:

```
[128 KB allocated to job] [128 KB free]
```

Freeing and Merging

- When the job frees its 128 KB block, the system checks if its buddy (the other 128 KB block) is also free.
- If so, it merges them back into a **256 KB block**, reducing fragmentation.

This revised example aligns with the buddy system's behavior and the splitting process described in the query, assuming the 200 KB request was a typo or misunderstanding.

Applying the Correct Process to the Original 200 KB Request

For completeness, let's apply the correct buddy system logic to the original **200 KB request**:

Scenario

- Job requests: **200 KB**.
- Block sizes: 128 KB, 256 KB, 512 KB, etc.

Allocation Process

1. **Determine the Required Block Size**:
   - 128 KB < 200 KB < 256 KB.
   - The smallest suitable block is **256 KB**.
2. **Allocation**:
   - If a **256 KB block** is available, it's allocated to the job directly.
   - If not, and a **512 KB block** is available:
     - The 512 KB block is split into two **256 KB blocks**.
     - One 256 KB block is allocated to the job.
     - The other 256 KB block is marked free.
3. **Outcome**:
   - The job gets a **256 KB block**, meeting its 200 KB need with some extra space (a common trade-off in the buddy system).

Visual Representation

- **Before Splitting**:

```
[512 KB free]
```

- **After Splitting and Allocation**:

```
[256 KB allocated to job] [256 KB free]
```

Conclusion

The **buddy system** in Linux is a fast and efficient memory allocation mechanism that manages memory in blocks sized as powers of two. It excels at providing large, contiguous memory blocks, making it ideal for **HPC applications** that demand high performance and low latency. The system allocates the smallest block that meets a request, splitting larger blocks when necessary and merging them when freed to maintain efficiency.

Key Points:

- For a **200 KB request**, the buddy system allocates a **256 KB block**, not a 128 KB block, as 128 KB is too small.
- Splitting occurs only to create blocks of the required size or larger, not smaller.
- A corrected example (e.g., a 100 KB request) shows splitting a 256 KB block into two 128 KB blocks, allocating one, and saving the other.

By leveraging this approach, Linux ensures that memory allocation is both rapid and robust, supporting the demanding needs of HPC workloads effectively.

File Systems in Linux

File systems manage data storage and access. In HPC, they must support massive data and parallel operations across nodes.

1. Types of File Systems

ext4 (Extended File System 4)

- **Features:** Reliable, good for single-node use.
- **HPC Role:** Fine for local storage, but not built for parallel access.

Example:
A node uses ext4 to store a small dataset for a single-threaded simulation.

NFS (Network File System)

- **Features:** Shares files over a network.
- **HPC Role:** Good for sharing data across nodes, but slow for high-speed parallel tasks.

Example:
An HPC cluster uses NFS to share a 10 GB input file across 5 nodes, but access slows if all nodes read at once.

Lustre

- **Features:** Parallel file system for HPC, handles huge data with high speed.
- **HPC Role:** Perfect for big, parallel I/O tasks like simulations.

Example:
100 nodes in a supercomputer use Lustre to access a 1 TB dataset simultaneously, with each node reading its chunk quickly.

2. File System Hierarchy

Linux's standard hierarchy includes:

- /bin: Binaries
- /etc: Config files
- /home: User files

In HPC:

- /scratch (often Lustre): High-speed storage for big data.
- /shared (often NFS): Shared resources across nodes.

Example:
A cluster mounts Lustre at /scratch for fast data access and NFS at /shared for common tools.

Linking Memory and File Systems in HPC

In HPC, memory and file systems work together:

- **Memory-Mapped Files:** Files are accessed as memory, speeding up I/O.
- **Parallel Access:** Lustre feeds data to memory across nodes efficiently.

Example:
A simulation maps a 5 GB file from Lustre into memory, letting nodes process it without slow disk reads.

Summary

- **Memory Management:**
  - Virtual memory isolates processes.
  - Paging optimizes memory layout.
  - Swapping handles overflow but slows HPC.
  - Buddy system allocates memory efficiently.
- **File Systems:**
  - ext4 for local use, NFS for sharing, Lustre for HPC parallel I/O.
  - Hierarchy adapts for HPC needs.

These concepts help HPC systems run fast and handle huge workloads effectively.